QUASAR: Cryptographic Intent-Aware Zoned Storage for Post-Quantum Workloads

Anonymous Submission

Abstract

Post-quantum services such as TLS terminators, KMS deployments, and audit systems write not only payload and logs, but also KEM artifacts, session-derived recovery records, key-wrap records, certificate metadata, and signatures whose lifetimes follow sessions, key epochs, rotations, and erase policy. Zoned Namespace SSDs expose append and reset primitives that can exploit lifetime structure, but DOGI, MiDAS, and SepBIT infer lifetime from LBA locality, frequency, and previous invalidations, which do not encode the protocol event that kills a post-quantum secret. This paper presents QUASAR, a cryptographic intent-aware placement architecture that adds a 32-byte lifecycle hint to the write path, preserves history-based placement for ordinary payload, and packs short-lived post-quantum objects by death cohort so epoch close can reset eligible secret-bearing zones. On an actual WD ZN540-class ZNS SSD, a same-path replay over six DOGI-style workload axes completes 72 policy rows with no failures; DOGI-style placement leaves 99,834 stale secret blocks, while QUASAR-DOGI hybrid leaves zero and reduces GC by 83.0%, and FAST-style Sysbench plus dynamic Exchange-, Varmail-, and Alibaba-like pressure traces show 92.3–99.9% GC reduction while preserving zero stale-secret exposure.

1 Introduction

Post-quantum cryptography is moving from a standards problem to a systems problem. Once secure services adopt ML-KEM, ML-DSA, and related post-quantum mechanisms, the write path of a TLS terminator, key-management service, or signed logging system begins to carry more than application payload. It also carries KEM ciphertexts, encapsulation metadata, session-derived recovery records, key-wrap records, certificate metadata, and signatures. These objects are not all alike: some are append-only audit records, some are public metadata, and some are short-lived secret-bearing records whose lifetime ends at session close, epoch advance, or key rotation.

Zoned Namespace SSDs are a natural substrate for such services because they expose the structure that log-structured systems already exploit: data is appended into zones and reclaimed by resetting whole zones. The placement problem is therefore crucial. If objects that die together are placed together, a zone reset can reclaim space with little or no valid-block copying. If short-lived and long-lived objects are mixed, the host must either copy residual live blocks during GC or leave expired data in place until a future cleaner selects the zone. Recent ZNS and log-structured placement systems, including SepBIT, MiDAS, and DOGI, attack this problem by inferring block lifetime from storage-visible history such as LBA, frequency, age, segment behavior, and prior invalidation time [3, 11, 15].

The central mismatch in post-quantum workloads is that the lifetime of cryptographic artifacts is often not hidden at all. The application or cryptographic runtime already knows that a session-derived recovery record will die at session close, that a KMS envelope belongs to an epoch, and that a certificate object follows a rotation policy. The storage device does not see this state. From the perspective of a history-based placement system, a KEM artifact may look like another small write near a hot payload record. From the perspective of the protocol, it belongs to a death cohort: a set of objects that become invalid together when the cryptographic state advances.

Table 1 gives the first-order distinction. The point is not that learned placement is useless. DOGI is designed for the regime where lifetime must be inferred from noisy storage behavior, and our own non-PQC controls preserve that advantage. The point is that post-quantum metadata introduces a different regime: the lifetime boundary is a semantic event defined above the block layer. Predicting this boundary from LBA and frequency is the wrong interface when a small lifecycle label can expose it directly.

This paper proposes QUASAR, a cryptographic intent-aware ZNS placement architecture. QUASAR follows one design rule: place data by when the cryptographic protocol will kill it, not by how the storage device has seen it before.

Table 1: The paper's core gap. Existing history-based placement systems observe storage history; QUASAR observes the small lifecycle signal that defines post-quantum death cohorts.

Policy	Main signal	Strength	PQC blind spot	Consequence
SepBIT-style	Write/invalidation history and group separation	Simple hot/cold-style lifetime grouping	Session close and key rotation are not encoded in history	Expired secrets remain mixed with payload or metadata.
MiDAS-style	Adaptive groups from historical invalidation patterns	Can keep WAF low on stable workloads	Stable storage history can hide protocol-driven burst invalidation	WAF may look good while stale secrets remain.
DOGI-style	Hybrid lifetime prediction and adaptive group organization	Strong general-purpose learned placement	MLP features do not include cryptographic epoch or intent	Pays prediction cost and still misses death cohorts.
QUASAR-DOGI hybrid	DOGI-like payload placement plus cryptographic lifecycle hints	Keeps history placement where it is useful and adds semantic placement for secrets	Requires correct and conservative hint handling	Makes secret-bearing zones reset-eligible at epoch boundaries.

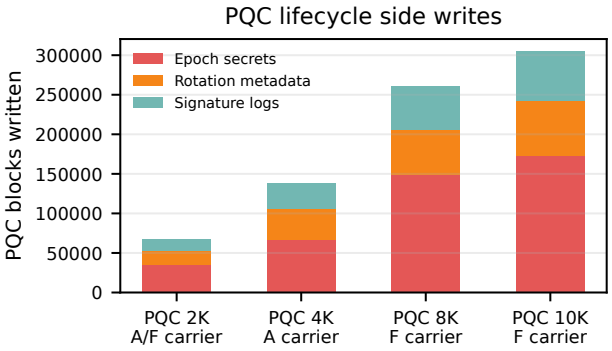


Figure 1: PQC lifecycle side writes on actual ZNS traces. The figure is workload characterization, not an evaluation result: post-quantum services add epoch secrets, rotation metadata, and append-only signatures whose lifetimes are defined by cryptographic protocol state rather than storage history.

Each write carries a compact hint containing intent class, expiration class, security class, epoch, cohort identifier, and size. The allocator maps short-lived secrets and KEM artifacts to epoch-secret zone families, maps rotation metadata to rotation families, keeps append-only signatures separate, and leaves ordinary payload on a history-based fallback. The deployable default is a hybrid: DOGI-style placement remains responsible for payload locality, while QUASAR handles the death cohorts that DOGI cannot observe.

The design also changes the security story of ZNS placement, but only within a strict boundary. Zone reset by itself is not a physical erase primitive, and NVMe sanitize/crypto-erase is not a per-zone command on a shared namespace. A shared-drive sanitize operation can destroy data far outside one epoch cohort. QUASAR therefore does not claim chip-off physical erasure by default. Its default security contribution is narrower and measurable: it aligns secret lifetime with zone-reset eligibility, reducing the host-visible stale-secret exposure window. For a stronger crypto-erase scope, we validate per-cohort encryption-key destruction; stronger hardware physical erasure would still require a dedicated namespace/-media pool or future hardware erase semantics whose blast radius matches the cohort being destroyed.

Our evaluation follows the structure of the DOGI/FAST workload axis rather than inventing a clean PQC-only trace. We use six DOGI-style workload families, YCSB-A/F pressure traces, dynamic Exchange-, Varmail-, and Alibaba-like pressure traces, and a FAST-style Sysbench-OLTP stressor. We explicitly separate easy negative controls from WAF-pressure results. On the actual ZNS path, the six-axis fairness matrix completes 72 rows with no failed policy runs. In that matrix, DOGI-style placement already has WAF near 1.0, but it leaves 99,834 stale secret blocks and issues no semantic zone resets. QUASAR-DOGI hybrid removes all stale secret blocks, issues 98 semantic zone resets, and reduces GC blocks by 83.0%. Under Sysbench pressure, the hybrid reduces DOGI-style GC blocks by 94.6% and removes 38,438 stale secret blocks. Under dynamic service pressure, it reduces GC by 92.3–99.9% while keeping stale secret blocks at zero.

This paper makes the following contributions:

- It identifies death cohorts as the missing placement unit for post-quantum storage workloads. Unlike hotness or LBA locality, a death cohort is defined by cryptographic protocol state.
- It presents QUASAR, a ZNS allocation architecture that uses a small lifecycle hint to separate short-lived secrets, rotation metadata, append-only signatures, and payload.
- It implements a trace-driven and actual-ZNS evaluation path with FIFO, SepBIT-style, MiDAS-style, DOGI-style, QUASAR, and QUASAR-DOGI hybrid policies.
- It shows that the strongest claim is not universal WAF dominance. Instead, QUASAR exposes a semantic lifetime signal, removes stale-secret exposure across PQC overlays, and reduces WAF/GC when zone pressure makes mixed cohorts expensive.

2 Background

2.1 ZNS Placement and Write Amplification

ZNS SSDs expose a storage interface in which each zone is written sequentially and later reset as a unit [10]. This interface removes part of the hidden device FTL policy and gives the host more control over placement. The benefit depends on grouping objects with compatible lifetimes. When a zone contains only expired objects, reset is cheap. When a zone also contains live objects, the host must copy those valid blocks elsewhere before reset, increasing write amplification factor (WAF).

Prior placement systems therefore focus on predicting block invalidation time. SepBIT separates data using historical invalidation behavior. MiDAS adapts group configuration to workload patterns. DOGI combines hot filtering, learned invalidation-time prediction, GC relocation policy, and dynamic group organization. The DOGI paper evaluates both static-skew workloads such as FIO Zipf and YCSB-A/F and dynamic workloads such as Varmail, Alibaba, and Exchange [3]. This is the right baseline family for QUASAR because these systems represent the strongest storage-history approach.

2.2 Host-Guided Placement Interfaces

QUASAR is a lifecycle-placement policy, not a claim that native ZNS is the only possible interface. Native ZNS is the right first substrate because the host directly controls zone append, zone close, valid-block migration, and reset timing. That makes WAF, reset eligibility, zone fill, and stale-secret exposure auditable in the evaluation. Flexible Data Placement (FDP) exposes a different host-guided interface: the host supplies placement handles while the device keeps a conventional block interface and performs internal grouping [9].

Table 2 gives the deployment boundary. FDP can carry the same semantic grouping signal, but limited placement handles require hashing, binning, or admission control, and device-internal reclaim makes stale-exposure accounting less direct. We therefore use ZNS for the main experiments and treat FDP as the natural deployment extension.

Table 2: QUASAR can target both native ZNS and FDP-like host-guided placement. The paper evaluates ZNS first because it exposes reset and migration accounting directly.

Interface	QUASAR mapping	Boundary
Native ZNS	Zone family from `intent`, `epoch_id`, and security class.	Host-visible append/reset accounting.
FDP	Placement handle from hashed intent, epoch/cohort, security, and tenant.	Device-internal reclaim is less observable.
Fallback block path	Payload or low-confidence writes use history placement.	No strong reset eligibility claim.

2.3 Post-Quantum Object Lifetimes

Post-quantum workloads introduce a different kind of lifetime signal. A KEM artifact or secret-bearing recovery record is not short-lived because it has a particular LBA pattern; it is short-lived because the cryptographic protocol invalidates it. A signature log may be append-only and long-lived, while a KMS envelope may die when an epoch closes. A certificate metadata object may follow a rotation period rather than a session boundary. These differences exist within the same service and often within the same burst of I/O.

Table 3 gives the placement consequence. A single hotness label cannot safely place these objects because two objects with similar write frequency may have opposite reclaim requirements. Conversely, two objects with different LBAs and different sizes may be ideal zone neighbors if they die at the same epoch boundary.

Threat model for stored PQC state. QUASAR does not require every TLS session key to be synchronously persisted. Many deployments correctly keep raw session secrets only in RAM or in an HSM. The target is narrower: services that already persist bounded PQC lifecycle state, such as KMS envelopes and key-wrap records, crash-recovery or session-ticket state, compliance/audit records, certificate and signature metadata, or constrained edge spill paths. If a deployment emits no secret-bearing or lifecycle-bound PQC writes, QUASAR falls back to payload placement and makes no security improvement claim.

2.4 Claim Boundary

QUASAR is not an oracle. It does not receive a future deletion timestamp for every object, and it does not assume that every

Table 3: Post-quantum objects have different lifetime and security behavior even when they are generated by the same service.

Object	Lifetime driver	Placement implication
KEM artifact	Session or epoch close	Group by epoch; reset quickly.
Secret-bearing record	Session close	Isolate from payload and logs.
KMS envelope	Key epoch	Pack with same epoch family.
Certificate metadata	Rotation policy	Use rotation-zone family.
Signature log	Retention policy	Append-only placement.
Payload	Application update pattern	Use history-based fallback.

hint is perfect. The hint expresses lifecycle class and epoch membership, not secret key material. Correctness is protected by a conservative epoch manager: a zone family is reset only after the manager confirms that all objects in that family have expired or have been safely migrated.

The security claim is also scoped. Zone reset makes a zone reusable and can reduce exposure, but it is not by itself NIST-style media sanitization. NIST SP 800-88 Rev. 2 defines sanitization around making target data access infeasible for the relevant effort level [7]; a ZNS reset only changes host-visible zone state unless the device also provides stronger erase semantics. Our physical device advertises crypto-erase and block-erase sanitize support, and the crypto-erase sanitize command path has been executed successfully. This validates that the destructive command path exists on the device; it does not make sanitize a safe per-zone operation for a shared namespace. The default claim is therefore reset eligibility, logical isolation, and exposure-window reduction. We validate one matching crypto-erase scope with per-cohort encryption-key destruction; a stronger hardware physical-erase claim would require a dedicated namespace/media pool or future per-zone erase semantics [8].

3 Motivation

3.1 History Signals Do Not Encode Death Cohorts

History-based placement asks: what did this block look like before? DOGI-style policies observe LBA, previous LBA, access interval, segment frequency, and prior placement state. These features are meaningful for ordinary payload updates because they capture hotness and locality. A post-quantum secret asks a different question: which objects will die together when the protocol state advances? The answer requires at least intent and epoch.

This distinction creates two failure modes. The first is semantic: the baseline leaves expired secret blocks in zones that are not safely resettable because the zone also contains payload or long-lived metadata. The second is performance: under sufficient free-zone pressure, mixed death cohorts force valid-block migration and raise WAF. The first failure can appear even when WAF is near 1.0; the second appears only in harder pressure workloads.

Figure 2 shows the diagnostic reason this paper does not use WAF alone as the motivation. On DOGI-compatible YCSB carriers, DOGI-style placement often keeps physical WAF close to 1.0 because the payload locality signal is still useful. At the same time, expired PQC secrets remain stranded because the storage-visible history does not encode epoch close. QUASAR-DOGI removes those protocol-dead secrets. It routes only lifecycle objects through the death-cohort path.

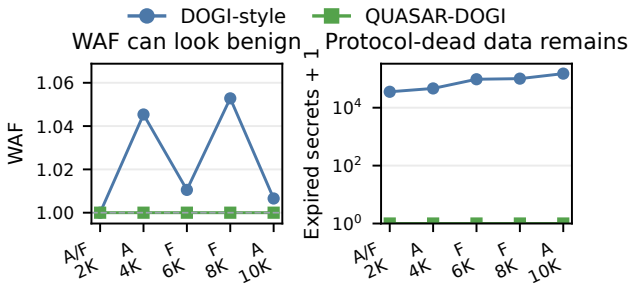


Figure 2: Motivation diagnostic: WAF is not enough to expose the PQC failure mode. DOGI-style placement can look benign on storage-visible WAF while still stranding expired PQC secrets; QUASAR-DOGI keeps payload on history placement but routes lifecycle objects by death cohort.

3.2 Easy Workloads Are Not Enough

A clean PQC-only epoch trace makes QUASAR look too good and is not a fair test of DOGI. Our evaluation therefore treats such traces as sanity checks only. The hard tests keep DOGI's normal locality signals and then add realistic PQC lifecycle writes. Concretely, the workload matrix contains: DOGI-style six-axis fairness, YCSB-A/F pressure, FAST-style Sysbench pressure, dynamic service pressure, and QUASAR-hostile robustness.

3.3 The Upper Bound Gives Design Requirements

DOGI uses NoDaP to turn an oracle gap into design requirements. QUASAR uses a narrower epoch upper bound for the same diagnostic purpose: by giving the oracle exact PQC death cohorts, we test whether the missing signal is protocol lifetime rather than another storage-history feature.

The upper bound yields three requirements. First, the placement path must expose protocol lifetime because LBA, frequency, and prior invalidations do not encode session close or key rotation. Second, exact cohort placement must be budgeted because a sparse epoch per zone can trade WAF for poor utilization and too many active families. Third, reset must be conservative because a wrong hint can hurt placement quality but must not discard live data. These requirements lead directly to QUASAR's lifecycle hint, admission/open-zone policy, and epoch-confirmed reclaim path.

The actual-ZNS results later separate these failure modes. In the six-axis fairness matrix, DOGI-style placement has WAF near 1.0 but leaves 99,834 stale secret blocks. In harder YCSB, Sysbench, Exchange, Varmail, and Alibaba-like rows, history-based placement pays positive GC while leaving stale secrets; the hybrid keeps payload on history placement and moves PQC secrets into resettable cohorts.

3.4 The Desired Design Shape

The design requirement is therefore not "replace DOGI everywhere." A pure semantic allocator can be worse for ordinary payload. The desired system uses history where history is the right signal and cryptographic intent where history is blind: DOGI-like placement for payload, QUASAR zone families for PQC lifecycle objects, and conservative fallback for unreliable hints.

4 Design

4.1 Overview

QUASAR is a host-side placement layer for ZNS devices. Figure 3 shows the three mechanisms that define the design: lifecycle labels enter the storage path, the allocator maps them to bounded zone families, and epoch reclaim resets a family only after safety is proven. Applications or cryptographic libraries produce lifecycle labels when they create PQC artifacts. These labels are attached to writes as `quasar_hint`. The allocator maps each write to a zone family. An epoch manager tracks which families are eligible for reset, migration, or fallback GC. Payload writes continue to use a history-based fallback, so the system does not discard the strengths of DOGI-style placement.

4.2 Design Requirements

QUASAR follows the three requirements from Motivation. Lifecycle labels expose protocol lifetime and route PQC objects to death-cohort zone families while payload stays on DOGI-style fallback. Admission control budgets exact placement by reserving scarce exact families for high-security secrets and sending sparse or uncertain writes to bins or overflow. Epoch-confirmed reclaim resets a family only after durable epoch close or residual migration; uncertain zones fall back to normal GC. The evaluation checks these requirements with stale-secret exposure rows, pressure traces, multi-tenant stress, bad hints, and sanitize-boundary tests.

4.3 Hint Schema

The hint is intentionally small. It contains only lifecycle metadata:

Listing 1: Minimal QUASAR hint schema.

```
enum quasar_intent {
  QUASAR_PAYLOAD = 0,
  QUASAR_EPHEMERAL_SECRET = 1,
  QUASAR_KEM_ARTIFACT = 2,
  QUASAR_SIGNATURE_LOG = 3,
  QUASAR_CERT_METADATA = 4,
};

struct quasar_hint {
  uint16_t intent;
  uint16_t expire_class;
  uint32_t security_class;
  uint64_t epoch_id;
  uint64_t cohort_id;
  uint64_t size_bytes;
};
```

The fields are lifecycle labels, not cryptographic secrets. `intent` separates secrets, KEM artifacts, signature logs, certificate metadata, and payload. `epoch_id` identifies the death cohort. `security_class` sets reset urgency and isolation. `cohort_id` is opaque to storage and can be a keyed hash, so raw session, tenant, or key identifiers need not cross the storage boundary.

Trust boundary. QUASAR assumes that high-priority lifecycle hints are emitted by trusted components such as the TLS terminator, KMS, logging service, or a privileged policy layer. Untrusted tenants do not directly grant themselves exact secret-zone priority. A deployment can enforce per-tenant exact-family quotas, tenant-local cohort namespaces, and admission limits before a hint reaches the allocator. If provenance is missing or confidence is low, the write is routed to overflow and becomes a performance/exposure tradeoff rather than a correctness risk.

4.4 Hint Delivery and Enforcement

The hint path is explicit about what is implemented and what is a deployment route. The prototype attaches `quasar_hint` in the replay record and user-space request context. A production stack can carry the same 32-byte label through a kernel-bypass request context, a file/extent metadata path, or an FDP placement handle. Standard POSIX `write()` does not magically carry this metadata; one of the following attachment points is required.

The enforcement rule is the same for all paths: storage does not try to infer whether a tenant is honest from raw `epoch_id`. A trusted service or policy layer authenticates the writer, applies quotas and family budgets, and emits a bounded hint. The allocator treats missing provenance as low confidence and routes the write to overflow.

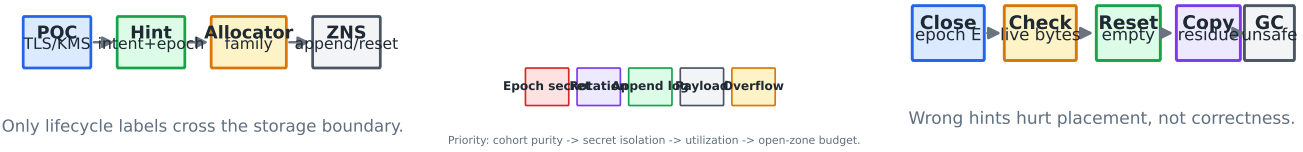
4.5 Zone Families

QUASAR allocates writes into a small set of zone families:

The key invariant is separation: the allocator keeps short-lived secrets out of long-lived payload and append-only log zones unless explicit pressure routes them through a conservative fallback. This is the mechanism that turns protocol lifetime into reset eligibility.

4.6 Allocation Procedure

For each write, QUASAR first classifies the object:



(a) Hint path: lifecycle labels, not secret bytes. (b) Zone-family split and admission priority. (c) Epoch reclaim and conservative fallback.

Figure 3: QUASAR design overview. Like DOGI's design figures, the figure separates the data path, placement organization, and reclaim rule. The storage path receives lifecycle labels, not secret material. Payload remains on a history-based fallback, while PQC secrets, rotation metadata, and uncertain writes are routed to separate zone families. The epoch manager resets a family only after proving that all objects in the family are expired or migrated; physical erase is a separate deployment path whose scope must match the cohort.

Table 4: Concrete hint delivery paths and trust enforcement points. Only trusted policy components can assign exact secret-zone priority; untrusted or unverifiable labels go to overflow.

Path	Carrier	Status and enforcement
Replay / user-space ZNS	Request context beside each write.	Implemented in the trace and physical replay path.
SPDK / xNVMe-style	Per-command request metadata.	Kernel-bypass deployment route; policy layer validates provenance before submission.
File/extent path	`xattr` or `ioctl` on files/extents.	Suitable for KMS records, logs, and certificate metadata.
`io_uring` path	Submission metadata or fixed-buffer side table.	Kernel route for asynchronous TLS/KMS writers.
FDP path	Placement handle from lifecycle family hash.	Maps exact families to scarce handles; bin or overflow when handles are exhausted.

Listing 2: Zone-family decision rule.

```
if intent in {EPHEMERAL_SECRET, KEM_ARTIFACT}:
    family = EPOCH_SECRET_ZONE(epoch_id, security_class)
elif intent == CERT_METADATA:
    family = ROTATION_ZONE(rotation_epoch(epoch_id))
elif intent == SIGNATURE_LOG:
    family = APPEND_LOG_ZONE
elif intent == PAYLOAD:
    family = DOGI_HISTORY_FALLBACK
else:
    family = OVERFLOW_ZONE
```

The allocator appends the write to the active zone for that family. If the active zone lacks capacity, the zone is closed and a new zone is allocated. When the active-zone budget is tight, QUASAR preserves exact placement for secret data first, bins lower-risk KEM artifacts or sparse epochs second, and routes uncertain writes to overflow. This priority order prevents a utilization-first policy from degenerating into arbitrary mixing.

Table 5: QUASAR zone families. Each family maps a lifecycle class to a reclaim rule, separating short-lived secrets from payload and append-only logs.

Family	Data	Reclaim rule
Epoch-secret	KEM artifacts, ephemeral secrets	Reset when epoch expires.
Rotation	Certificates, key metadata	Reset on rotation boundary.
Append-log	Signatures, audit records	Normal append-log reclaim.
Payload	Application payload	History-based fallback.
Overflow	Missing or low-confidence hints	Conservative GC or migration.

4.7 Admission Control and Open-Zone Budget

Exact epoch placement is useful only when the device can afford the active families it creates. Real ZNS devices bound the number of open and active zones; the evaluated device exposes a limit of 13 for both open and active zone accounting in our robustness experiments. QUASAR therefore admits exact epoch-secret families only when the expected epoch bytes justify a dedicated zone and the active family count remains below the budget.

When a sparse or multi-tenant workload would exceed the budget, QUASAR degrades in a fixed order. It first keeps exact families for high-security secrets, then bins lower-risk or sparse cohorts by coarse epoch ranges, then routes low-confidence writes to overflow. This policy makes the cost vis-

Table 6: Main QUASAR policy parameters. These knobs expose the cost of exact placement: active-zone budget, sparse-epoch binning, residual migration, hint confidence, and tenant isolation are workload/device inputs, not hidden oracle information.

Parameter	Role
Active-family budget	Caps concurrently open/active death-cohort families; robustness runs use the device-reported 13-zone budget.
Minimum epoch fill / bin width	Decides whether sparse epochs receive exact zones or coarse cohort bins.
Residual threshold / copy budget	Bounds live-byte migration before reset; strict mode raises this budget to force zero waiting.
Hint confidence	Sends missing, untrusted, or inconsistent hints to overflow instead of exact secret zones.
Tenant mode	Uses tenant-local bins and quotas when reset-time tenant mixing is not acceptable.

ible: exact placement maximizes exposure reduction, binning trades some exposure precision for zone budget, and overflow preserves correctness when the hint path is unreliable.

4.8 Epoch Reclaim

At epoch close, the epoch manager evaluates all zone families associated with that epoch:

Listing 3: Conservative epoch reclaim.

```
on epoch_expire(epoch_id):
  for zone in zones_owned_by(epoch_id):
    if live_bytes(zone) == 0:
      reset(zone)
    elif live_bytes(zone) <= residual_threshold:
      migrate_live_bytes(zone)
      reset(zone)
    else:
      downgrade_to_normal_gc(zone)
```

This rule makes wrong hints a performance problem rather than a correctness problem. A family is reset only after the epoch manager confirms that every object in the family has expired or has been safely migrated. If the system cannot prove safety after a crash or hint inconsistency, it routes the zone to normal GC.

4.9 Crash and Recovery Invariant

QUASAR cannot keep epoch-to-zone ownership only in volatile memory. Each zone family has durable metadata that records its family identifier, epoch, intent class, and reset generation. After a crash, recovery scans this metadata, rebuilds the family-to-zone map, replays durable epoch-close records, and resets only zones whose epoch close is known to be durable. Uncertain zones are downgraded to normal GC. This conservative rule is central to the security story: a bad hint or incomplete recovery can reduce placement quality, but it must not cause a live object to be discarded.

4.10 Deployment Modes

The deployable design is not a single universal knob. The default mode is QUASAR-DOGI hybrid: history-based placement for payload and death-cohort placement for PQC secrets. Tenant-isolation mode uses tenant-local bins when reset-time tenant mixing is unacceptable. Strict-residual mode copies delayed-expiry stragglers to force zero final secret waiting. Overflow mode handles missing or low-confidence hints. FDP mode maps `hash(intent, epoch/cohort, security, tenant)` to placement handles and reuses the same admission, binning, and overflow policy when handles are scarce. These modes make the cost visible instead of pretending that perfect epoch alignment is free.

5 Implementation

5.1 Trace and Replay Infrastructure

We implemented QUASAR as a trace-driven allocator and actual-ZNS replay framework. The trace format records writes, expirations, object identifiers, sizes, LBA-like payload location, intent, epoch, security class, tenant, and confidence. The same trace is replayed through FIFO, SepBIT-style, MiDAS-style, DOGI-style, QUASAR, and QUASAR-DOGI hybrid policies. This keeps timing and object lifetimes identical across policies.

The implemented deployment hook is intentionally small: a TLS terminator, KMS, logging component, or privileged policy layer emits the 32-byte `quasar_hint` when it writes a KEM artifact, secret-bearing recovery record, certificate record, key-wrap record, or signature log. In the prototype, this hint is carried in the JSONL trace and in the user-space request context that drives the physical replay. This is equivalent to a kernel-bypass SPDK/xNVMe request-context design in terms of allocator input, but it is not a full SPDK poll-mode implementation. Kernel integration would use one of the attachment points in Table 4; untrusted writers cannot directly allocate exact secret families because the policy layer validates provenance and quota before the allocator sees the hint.

The workload generators produce three classes of traces. First, DOGI-paper-shaped traces mirror the workload axes described by DOGI: FIO Zipf, YCSB-A, YCSB-F, Varmail, Alibaba, and Exchange. Second, pressure traces increase PQC density and reduce free-zone slack to force GC. Third, hostile traces introduce missing hints, wrong epochs, tenant pressure, and stragglers. The implementation also includes a real FIO iolog converter that maps benchmark-issued FIO writes

into the same QUASAR JSONL schema before adding PQC lifecycle side writes. To close the application-trace realism gap, we additionally captured a real sysbench fileio block trace with `blktrace` while a liboqs KMS/audit side writer persisted PQC lifecycle records. That artifact proves the trace-collection path for real application I/O plus PQC side writes; it is not presented as a full SPDK/ZenFS end-to-end latency result.

5.2 Baseline Implementations

The same-path comparison implements FIFO, SepBIT-style, MiDAS-style, and DOGI-style policies in the replay framework. These implementations are intentionally described as style-compatible baselines because they share the same physical packed-ZNS path as QUASAR. The artifact set also contains exact public DOGI/MiDAS/SepBIT runs in their native environments. We keep these exact runs separate from the same-path results because their units and internal segment models are not identical to the packed replay path.

5.3 Physical ZNS Path

The actual-ZNS experiments use a Western Digital ZN540-class ZNS NVMe SSD mounted with `zonefs`. The replay writes zone files sequentially and issues reset-like truncation or helper operations according to the policy plan. This path measures full-suite append/reset accounting, but it includes zonefs helper overhead. We therefore report it as actual-ZNS replay overhead, not as final production p99 latency.

To check the lower-overhead command path, we also implemented an xNVMe raw ZNS latency probe. The local xNVMe backend uses Linux NVMe ioctl synchronous commands. It bypasses the zonefs helper path and completes 4,096 4KiB appends with p99 append latency of 26,064 ns. This is not an SPDK poll-mode result, but it bounds the helper-path caveat.

5.4 Security Capability Check

The physical device reports sanitize capability with crypto-erase and block-erase support. We executed an NVMe crypto-erase sanitize command on the device and observed successful completion in the sanitize log. This validates the destructive command path, but it is not a per-zone or per-epoch erase primitive for a shared namespace. The evaluation keeps this separate from zone reset: QUASAR makes secret-bearing zones eligible for host-visible reset, and the per-cohort key-isolation artifact validates a cohort-scoped crypto-erase path by destroying one epoch key while preserving unrelated cohorts. Hardware physical erasure would still require an isolated namespace/media pool or hardware erase semantics with cohort-sized scope.

6 Evaluation

6.1 Evaluation Setup

Evaluation questions. The evaluation answers five questions:

1. Does QUASAR avoid relying on an easy PQC-only trace?
2. Does QUASAR remove stale-secret exposure on DOGI-style workload axes?
3. Under pressure, does QUASAR reduce WAF and GC relative to history-based placement?
4. What overhead does semantic placement introduce?
5. What security claim is supported by actual device capability?

Evaluation platform. We evaluate placement with trace-driven replay and an actual ZNS device: a Western Digital ZN540-class NVMe SSD exposed as `/dev/nvme0n1` and mounted through `zonefs`. The replay model uses 4KiB logical blocks and a 275,712-block zone capacity. As in DOGI's methodology, the simulator/replay path gives broad policy coverage, while the physical path validates that the planned append/reset schedule executes on a zoned device. Because the full-suite path includes zonefs helper overhead, we report its latency as overhead accounting rather than production SPDK p99 latency.

Baselines. We compare against FIFO, SepBIT-style, MiDAS-style, and DOGI-style placement. These are same-path policies in our simulator and actual-ZNS replay. We also keep exact public DOGI/MiDAS/SepBIT runs as separate evidence because their internal units are not directly interchangeable with our packed ZNS replay.

Workloads. The fairness matrix uses six DOGI-style axes: FIO Zipf, YCSB-A, YCSB-F, Varmail, Alibaba, and Exchange. Headline WAF rows keep these locality signals and add PQC lifecycle side writes through high-ratio YCSB-A/F, FAST-style Sysbench-OLTP pressure, and dynamic Exchange/Varmail/Alibaba-like traces. Pure PQC epoch streams are sanity checks, not headline evidence. Hostile robustness uses tenant mixing, missing hints, wrong hints, and stragglers; real FIO iolog and a sysbench fileio `blktrace` capture with concurrent liboqs PQC KMS/audit writes check trace realism.

Metrics. We report WAF, GC write blocks, stale secret blocks, semantic zone-reset count on the physical device, secret bytes waiting for zone reset, zone utilization, live physical zones, append/reset latency, and C-level placement-decision overhead. A stale secret block is an expired secret-class logical block in a family that is not safely reset-eligible because live data remains or epoch close is unproven. Exposure artifacts report stale-secret block-seconds; physical tables use final and representative stale-block counts for auditability. WAF

alone is insufficient because a policy can keep WAF near 1.0 while leaving expired PQC secrets in mixed zones. For example, the E4 exposure timeline records max stale-secret block-seconds of 69,391,500 for FIFO and DOGI-style placement, while QUASAR remains at zero block-seconds and zero final stale-secret blocks.

Claim gate. The workload hardness rule is: DOGI-friendly first, PQC-hostile second. A headline WAF/GC row must preserve storage-visible structure that helps history-based placement—hot/cold skew, overwrite locality, segment reuse, or phase changes—and then add PQC objects whose invalidation follows protocol state rather than LBA history. Pure PQC-only traces, oracle-expiration traces, and low-pressure rows are sanity checks or exposure evidence.

The matrix passes all nine gates: fairness, negative controls, pressure rows, main-claim eligibility, and hostile robustness. The artifact set has seven eligible YCSB pressure rows, nine YCSB baseline-complete rows, three dynamic eligible rows, and DB pressure eligibility. Rows with WAF near 1.0 are interpreted as semantic exposure results; WAF/GC claims come only from rows where history-based policies pay positive GC or copied-live-block cost.

Epoch upper bound. DOGI uses NoDaP as an oracle-like target for lost lifetime information. We use a narrower upper bound: an epoch oracle that sees exact death time at write time. It is not deployable and is not counted as QUASAR; it tests whether the workload is placeable once the protocol-lifetime signal exists.

On the mixed sanity trace, DOGI-style placement has WAF 1.791 and 484,967 GC blocks, QUASAR has WAF 1.010 and 6,331 GC blocks, and the epoch oracle has WAF 1.000 and zero GC blocks. On the stress-rekey trace, DOGI-style placement has WAF 2.031, QUASAR has WAF 1.019, and the oracle has WAF 1.003. These synthetic rows are bounds, not headline evidence: most of the gap comes from whether the allocator can observe the protocol lifetime boundary. We therefore do not use the oracle as a main evaluation figure; the plotted headline evidence below compares all same-path baselines on physical-ZNS pressure rows.

6.2 PQC Lifecycle Pressure

DOGI-favorable control. The non-PQC control verifies that the hybrid preserves history placement. On the six DOGI-style axes with no PQC side writes, DOGI-style placement and the QUASAR-DOGI hybrid produce the same WAF because payload stays on the history policy. DOGI-style placement beats FIFO and pure QUASAR on all six workloads, and no stale secret blocks exist. This keeps the claim boundary clear: QUASAR is not a replacement for learned placement when no cryptographic death-cohort signal exists.

Six-axis fairness matrix. The six-axis actual-ZNS fairness matrix completes 72 rows with zero failures. Table 7 reports the aggregated policy results. DOGI-style placement is already strong on WAF, which is expected on an easy fairness matrix. However, it leaves 99,834 stale secret blocks and performs no semantic zone resets. QUASAR-DOGI hybrid leaves zero stale secret blocks, performs 98 semantic zone resets, and reduces GC blocks from 625 to 106.

Table 7: Same-path actual-ZNS fairness matrix over six DOGI-style axes at pqc2000. The easy matrix keeps DOGI-style WAF near 1.0, but only QUASAR-DOGI hybrid removes stale secrets while reducing GC.

Policy	WAF	GC	Stale secrets	Resets
FIFO	1.0040	3,903	99,141	0
SepBIT-style	1.0023	2,241	99,822	0
MiDAS-style	1.0006	581	99,898	0
DOGI-style	1.0006	625	99,834	0
QUASAR	1.0011	1,121	0	98
QUASAR-DOGI hybrid	1.0001	106	0	98

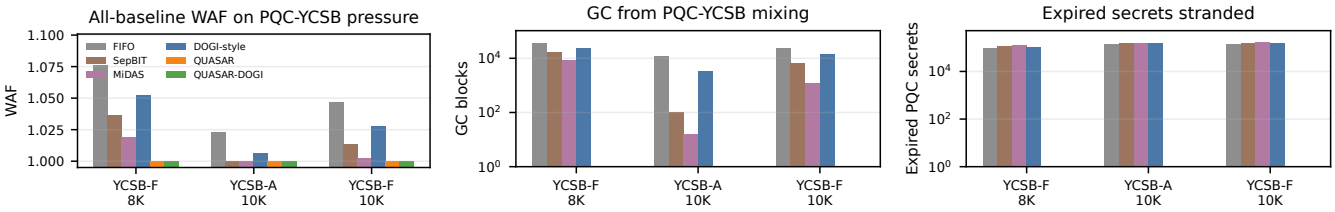
The WAF gain here is deliberately modest: 0.1% relative to DOGI-style. The row is a semantic result, not a performance headline: storage-history placement can have good WAF and still fail to make expired PQC secrets resettable.

YCSB carrier pressure. These rows use YCSB only as a storage-locality carrier. The target phenomenon is PQC lifecycle pressure: epoch secrets, KEM artifacts, rotation metadata, and append-only signatures are inserted beside ordinary payload, then invalidated by protocol state. Figure 4 reports representative physical ZNS replays. Across the plotted pressure rows, DOGI-style placement pays 3,357–24,289 GC blocks and leaves 46,266–150,221 expired PQC secret blocks. The hybrid removes stale secret blocks in all rows and reduces GC to zero in these representative replays.

The p10000 rows bound the WAF claim. Dense YCSB overlays can keep DOGI-style WAF close to 1.0, yet the stale-secret gap remains large. WAF improvement appears under pressure; exposure improvement appears across PQC overlays.

Multi-seed ratio sweep. To check that the pressure result is not a single-seed artifact, we run a three-seed ratio sweep over the six DOGI-style workload families at 0%, 5%, and 20% PQC overlay ratios, for 54 comparisons. The hybrid matches DOGI-style at 0% PQC, is WAF-negative at 5% PQC (-1.1% mean WAF, -12.4% mean GC) while avoiding 2,342 stale secret blocks per seed, and becomes positive at 20% PQC (5.0% mean WAF and 38.3% mean GC improvement) while avoiding 10,021 stale secret blocks per seed. Thus low-ratio overlays are exposure evidence, not WAF wins; WAF/GC benefits appear once protocol-lifetime objects create enough placement pressure.

Service carrier pressure. Figure 5 reports harder service carriers with the same PQC lifecycle side writes. Sysbench is included as DB-backed PQC service pressure rather than as a generic DB claim. The hybrid reduces DOGI-style GC from



(a) WAF: all baselines on PQC-YCSB pressure.

(b) GC blocks: history-based policies copy mixed cohorts.

(c) Expired PQC secrets: baselines strand protocol-dead data.

Figure 4: PQC lifecycle pressure on YCSB carriers, comparing all same-path baselines. The carrier supplies DOGI-friendly update locality; the side writes supply PQC objects whose invalidation follows protocol state. QUASAR and the hybrid keep payload on history placement and route PQC objects by death cohort, eliminating stale-secret exposure while reducing WAF/GC on pressure rows.

11,256 to 609 blocks and removes 38,438 expired PQC secret blocks. On Exchange-, Varmail-, and Alibaba-like carriers, the hybrid reduces GC by 92.3–99.9% and keeps stale secrets at zero.

These rows are the strongest performance evidence because all non-QUASAR baselines complete but pay visible GC and leave stale secrets. They also show why the hybrid is the default deployable policy: payload placement remains history-based, while secrets are placed by protocol cohort.

6.3 Mechanism and Baseline Analysis

Component ablation. Figure 6 is the metric-bearing ablation. History-only placement is the right first row because it gives DOGI-style placement all storage-visible signals but no protocol lifetime. Adding lifecycle hints removes stale secrets, but pure semantic grouping can still leave GC cost on DB-like payload. Adding the DOGI payload fallback removes that remaining cost. Admission and residual migration are not free performance wins; they are boundary mechanisms for open-zone pressure and stragglers, which Figure 7 exposes directly. Table 8 therefore does not repeat the plotted values; it records the design decisions implied by the ablation.

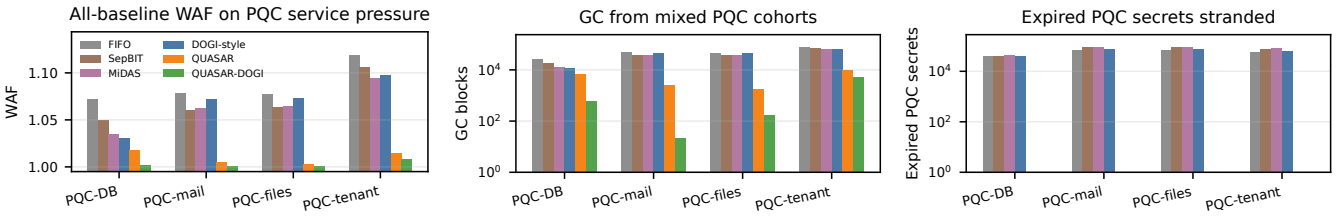
Exact public baseline sanity. The main tables use same-path replay so FIFO, SepBIT-style, MiDAS-style, DOGI-style, QUASAR, and the hybrid share trace timing, packed physical-ZNS model, and reset accounting. We also ran public baseline artifacts in their native environments. The public-DOGI parity audit records five direct checks: null_blk/ZenFS, a six-workload compact physical-ZNS suite, Alibaba p8000 compact pressure, DOGI/Greedy/CostBenefit selector variants, and Alibaba p8000 original-LBA pressure. Public DOGI reports WAF 2.993, 2.804, and 3.212 on the dynamic pressure checks; public MiDAS completes compact FIO and PQC representatives with WA 1.000 and 1.250; public SepBIT completes a PQC pressure representative with WA 2.430. These runs are sanity evidence, not apples-to-apples WAF numbers. They sharpen the boundary: MiDAS is strong on at least one compact representative, so the claim rests on death-cohort exposure and same-path pressure evidence, not universal baseline collapse.

6.4 Robustness and Overhead

Open-zone and hint robustness. The hardest QUASAR cases are not clean epochs; they are wrong or missing hints, many tenants, and delayed-expiry stragglers. On YCSB-A p4000, the clean hybrid has WAF 1.0000, zero stale secrets, and 28 semantic resets. With 5% missing hints, WAF remains 1.0000 but 3,361 stale secret blocks appear because some secret writes fall back to nonsemantic placement. With 5% wrong epochs, this trace still ends with zero stale secrets but increases reset activity to 112 resets. With 5% stragglers, exact secret-group packing can exceed the device open-zone budget; epoch-bin fallback completes within the budget but leaves waiting secrets, while epoch-bin plus residual migration reaches zero waiting at WAF 1.7098 after copying 234,549 residual blocks.

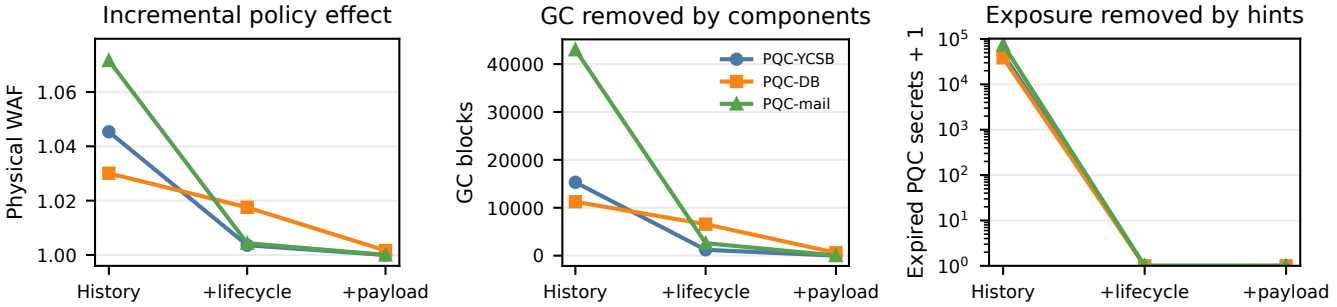
The residual fallback frontier shows why strict exposure mode cannot be the default. Exchange-like and Sysbench-like representatives can reach zero final secret waiting with physical WAF 1.0182 and 1.2161, respectively. YCSB-F p8000 reaches strict zero waiting only at physical WAF 3.5535 after migrating 1,168,095 residual blocks. QUASAR therefore exposes low-overhead, balanced, and strict-zero-wait profiles rather than hiding this tradeoff behind a single policy.

FDP handle-pressure model. To make the FDP deployment story concrete, we map the same QUASAR zone families from a mixed PQC trace onto a fixed number of FDP-style placement handles. This is a trace-driven handle model, not a physical FDP device result. The trace creates 86 QUASAR families. With only 8 handles, every occupied handle collides and family purity is 0.888; with 64 handles, family purity rises to 0.961 and intent purity to 0.982, with 1.76 families per occupied handle on average. At 128 handles, family purity reaches 0.980. Figure 8 therefore supports FDP as a deploy-



(a) WAF: all baselines on PQC service pressure. (b) GC blocks: mixed PQC cohorts create copy cost. (c) Expired PQC secrets: baselines leave stale data.

Figure 5: PQC lifecycle pressure breadth, comparing all same-path baselines. The x-axis names the service carrier, but the measured failure is PQC-specific: expired secret cohorts remain mixed with live payload under history-based placement. QUASAR and QUASAR-DOGI keep those cohorts reset-eligible while preserving payload placement on the history fallback.



(a) Physical WAF across component stages. (b) GC cost removed by lifecycle and payload split. (c) Expired PQC secrets after epoch close.

Figure 6: PQC lifecycle-signal ablation in the style of DOGI's component analysis. Moving from history-only placement to lifecycle hints removes expired-secret exposure; adding the DOGI-style payload fallback recovers payload placement and removes the remaining GC in the default hybrid. The x-axis is the implementation stage, not a new workload family.

ment carrier, but also shows why admission, hashing, and epoch binning remain part of the design when handle count is scarce.

Prototype overhead. QUASAR adds semantic reset work, but its decision path is cheap. In the actual-ZNS overhead suite, the hybrid issues 318 more semantic reset operations than DOGI-style. Zonefs-helper throughput is lower than DOGI-style by this accounting, but helper latency includes user-space append/truncate overhead. In the C-level placement-decision benchmark, DOGI-style feature extraction plus a small MLP costs 2,397.8 ns/write, QUASAR hint routing costs 16.8 ns/write, and the hybrid costs 1,178.8 ns/write.

The xNVMe probe provides a lower-overhead command-path sanity check: 4,096 4KiB zone appends complete with p99 latency of 26,064 ns and throughput of 181.79 MiB/s. The probe is a Linux NVMe ioctl path, not SPDK poll-mode, so it is reported as a bound rather than a final production datapoint.

Real application block trace. We also captured a sysbench fileio block trace while a liboqs KMS/audit side writer persisted PQC lifecycle records. The run traced `/dev/sdc2` for 8.0 s, completed 64 PQC sessions and 192 PQC audit records with successful KEM/signature checks, and produced 194,570 blkparse event lines including 155,360 write events. This closes the real-application trace realism gap for the artifact set, but it is not a SPDK/ZenFS p99 latency result and not a ZNS placement comparison.

6.5 Security and Reproducibility

Security boundary. The physical device advertises crypto-erase and block-erase sanitize support. We executed crypto-erase sanitize and observed successful completion in the sanitize log. This validates the destructive command path, not a per-zone erase mechanism. On a shared namespace, invoking sanitize for one PQC epoch would risk destroying unrelated payload or tenant data. The default result is therefore conservative: QUASAR reduces stale-secret exposure by aligning secret lifetime with reset eligibility. To close the erase-scope objection without using shared-namespace sanitize as an epoch primitive, we add a per-cohort DEK artifact: destroying the `epoch-4` key makes 32 target-cohort records inaccessible, preserves 224 unrelated records, rejects 32/32 wrong-key decryptions, and calls no sanitize command. This is cohort-scoped crypto erase by key destruction, not proof that zone reset physically erases NAND. We do not use sanitize timing to claim foreground service latency.

Reproducibility. The artifact manifest records 46 evidence

Table 8: Design checkpoints from the component ablation. The measured values are shown in Figure 6; this table records the allocator decision each checkpoint justifies.

Component	Checkpoint	Allocator decision
History only	Storage-visible history does not encode cryptographic death cohorts.	Do not place secret-class PQC objects by history-only grouping.
Lifecycle hints	Intent and epoch labels carry the missing lifetime boundary.	Route PQC artifacts by death cohort and security class.
Payload fallback	Ordinary payload still benefits from history-based placement.	Use the hybrid: QUASAR for PQC lifecycle objects and DOGI-style fallback for payload.
Admission	Exact epoch zones are useful only when fill and active-zone budget justify them.	Enable binning and overflow only under sparse or budget-limited epochs.
Residual	Strict zero-wait exposure control can require copying stragglers.	Expose balanced and strict profiles instead of hiding residual copy cost.

Figure 7: Open-zone and hint robustness, matching DOGI's configuration-sensitivity role. Exact epoch placement can exceed the open-zone limit, binning fits the limit but leaves waiting secrets, and residual migration removes waiting secrets at explicit physical-WAF cost.

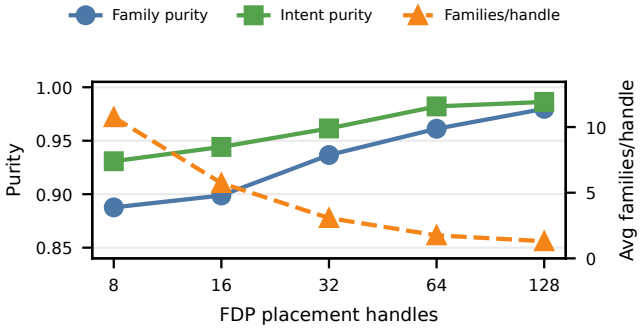


Figure 8: Trace-driven FDP placement-handle pressure. More placement handles preserve QUASAR family and intent purity, but small handle counts collide many death cohorts into the same handle. This is the FDP analogue of the open-zone budget problem, not a physical FDP performance result.

artifacts, their roles, abbreviated hashes, and regeneration com-

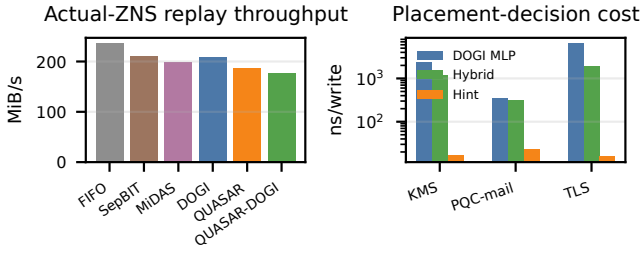


Figure 9: Prototype overhead in the role of DOGI's prototype figure. Zonefs replay is an accounting bound, while the isolated C-level benchmark shows lifecycle-hint routing is much cheaper than DOGI-style feature extraction and MLP scoring.

mands. Validation finds zero hash mismatches, and the local acceptance report passes 44 of 44 gates. A separate completion audit records the boundary explicitly: the scoped claim is ready, but the broader production-grade goal remains open because full public-DOGI parity, SPDK/ZenFS latency, physical FDP replay, and device diversity are not yet complete. These checks make the scoped evidence auditable without turning it into an overclaim.

7 Related Work

Learned and heuristic placement for log-structured storage. SepBIT, MiDAS, and DOGI improve log-structured placement by predicting or adapting to invalidation behavior [3, 11, 15]. SepBIT separates blocks by inferred invalidation time, MiDAS adapts group sizes and counts to workload dynamics, and DOGI uses oracle-guided analysis to motivate hot filtering, learned lifetime prediction, GC relocation policy, and dynamic group organization. These systems are the right comparison class for QUASAR because they represent the strongest storage-history view of placement.

QUASAR is not an argument that learned placement is generally unnecessary. For ordinary payload and workloads whose lifetime is not visible to the application, history is the right signal and DOGI-style placement can outperform semantic grouping. QUASAR targets a narrower regime where lifetime is not hidden from the software stack: a KEM artifact, secret-bearing recovery record, or KMS envelope dies because a protocol epoch closes. In that regime, prediction spends work approximating a label that the application already has, and a misplacement can leave expired secret bytes in a long-lived zone.

Zoned storage and host-guided placement. ZNS and related host-managed SSD interfaces expose placement and reclaim decisions to the host [10, 14, 16]. Systems such as ZenFS show how a host-side stack can map application writes onto zone append and reset operations. QUASAR builds on this host-managed direction but changes the placement key: instead of grouping only by hotness, stream, or predicted invalidation time, it groups secret-bearing writes by protocol death cohort.

Flexible Data Placement provides a different host-guided interface without forcing a full ZNS software stack [9]. QUASAR can target native ZNS by choosing zone families directly, or FDP-like devices by mapping intent and epoch to placement handles. This makes QUASAR a semantic placement policy rather than a device-specific API proposal.

Benchmarks and trace methodology. FAST storage papers typically evaluate both synthetic controls and macro workloads. DOGI uses FIO Zipf, YCSB-A/F, Varmail, Alibaba, and Exchange-like traces; our workload plan preserves these axes and adds PQC lifecycle overlays. FIO, YCSB, and Sysbench remain important because they provide recognizable hot/cold, update-heavy, and DB-style pressure points [1, 2, 4]. The paper therefore treats pure PQC epoch streams only as sanity checks and uses DOGI/FAST-shaped pressure rows for headline claims.

Post-quantum systems. Open Quantum Safe, liboqs, and oqs-provider provide software paths for deploying post-quantum algorithms in applications and TLS stacks [12, 13]. NIST's ML-KEM and ML-DSA standards make these deployments concrete targets rather than hypothetical future algorithms [5, 6]. Most systems work around PQC focuses on cryptographic correctness, interoperability, CPU overhead, and network handshake cost. QUASAR instead studies a storage side effect: lifecycle-heavy cryptographic metadata creates a placement signal that the block layer and device do not naturally observe.

Secure erase and sanitization. Storage sanitization and crypto-erase mechanisms provide device-specific ways to destroy data or keys [7, 8]. QUASAR does not replace these mechanisms and does not claim that zone reset alone physically erases NAND. It solves a placement precondition for lower exposure: expired secrets are isolated into zone families that can be reset without copying unrelated live data. Strong physical erasure is a separate hardware and deployment question. In particular, NVMe sanitize/crypto-erase may be device- or namespace-scoped rather than zone-scoped, so it is only compatible with QUASAR's epoch boundary when the deployment isolates the target cohort into a matching namespace, media pool, or encryption-key domain.

8 Discussion and Limitations

QUASAR is not a universal WAF optimizer. Easy rows deliberately show DOGI-style WAF near 1.0; there, QUASAR's benefit is exposure reduction. The scoped claim is that PQC death cohorts are invisible to history-based placement and become costly under pressure, not that QUASAR wins every trace.

Correctness, trust, and tenancy. Wrong or missing hints must not cause data loss: QUASAR resets only after epoch-close confirmation; otherwise it migrates residuals, downgrades to normal GC, or sends low-confidence writes to overflow. High-priority secret placement is emitted by trusted TLS/KMS/logging or policy components, with opaque cohort IDs, tenant-local namespaces, quotas, and admission limits to prevent priority inflation.

Resource tradeoffs are explicit. Dedicated epoch zones can waste space, and strict zero-wait residual handling can be expensive on hostile rows such as YCSB-F p8000. The default hybrid reserves exact placement for high-security cohorts, bins sparse cohorts, sends uncertain writes to overflow, and makes strict tenant/residual modes opt-in.

ZNS and FDP deployment. QUASAR should be read as a protocol-lifetime placement policy, not as a claim that every deployment must expose native ZNS. Native ZNS gives the paper direct host-visible append, reset, fill, and migration accounting. FDP can carry the same grouping signal through placement handles, but handle scarcity, hashing collisions, and device-internal reclaim reduce observability. A production deployment can therefore choose native ZNS for host-managed reset or FDP for lower integration friction, while keeping the same admission and overflow rules.

Device and deployment boundaries. Zone reset alone is not physical NAND erasure. Strong erasure requires semantics whose blast radius matches the cohort being destroyed. A shared-namespace NVMe sanitize/crypto-erase command

can be too broad: it may invalidate unrelated payload or tenant data and therefore cannot be treated as a per-zone epoch cleanup operation. Our per-cohort DEK artifact demonstrates one deployable crypto-erase scope: destroying one cohort key makes only that cohort cryptographically inaccessible while other cohorts remain decryptable. The physical evidence uses one WD ZN540-class device, zonefs replay, liboqs/OpenSSL traces, a TLS socket trace, a Sysbench/MySQL execution gate, per-cohort key-isolated crypto erase, and an xNVMe probe. Exact DOGI/MiDAS/SepBIT runs stay separate because their units differ from packed actual-ZNS replay. We report host-visible resets/utilization and cohort-scoped crypto-erase feasibility, but not chip-off erasure, internal wear telemetry, full application p99 latency, real DB block traces, FDP hardware behavior, hardware per-zone erase, or full SPDK poll-mode replay.

9 Conclusion

Post-quantum services can write KEM artifacts, bounded secret-bearing records, signatures, metadata, and payload together, but these objects die for different protocol reasons. QUASAR exposes that lifecycle signal as a small placement hint, keeps DOGI-style history placement for payload, and packs short-lived PQC objects by death cohort. Actual-ZNS experiments support the scoped claim: QUASAR removes stale-secret exposure across PQC overlays and reduces GC/WAF when pressure makes mixed cohorts expensive.

References

[1] J. Axboe. fio: Flexible i/o tester. https://fio.readthedocs.io/, 2024.

[2] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears. Benchmarking cloud serving systems with YCSB. In *Proceedings of the ACM Symposium on Cloud Computing*, 2010.

[3] J. Kim, S. Oh, J. Kim, J. Kim, J. Park, S. Lee, and S. H. Noh. DOGI: Data placement with oracle-guided insights for log-structured systems. In *Proceedings of the 24th USENIX Conference on File and Storage Technologies*, 2026.

[4] A. Kopytov. sysbench: Scriptable database and system performance benchmark. https://github.com/akopytov/sysbench, 2024.

[5] National Institute of Standards and Technology. FIPS 203: Module-lattice-based key-encapsulation mechanism standard. https://csrc.nist.gov/, 2024.

[6] National Institute of Standards and Technology. FIPS 204: Module-lattice-based digital signature standard. https://csrc.nist.gov/, 2024.

[7] National Institute of Standards and Technology. SP 800-88 Rev. 2: Guidelines for media sanitization. https://csrc.nist.gov/pubs/sp/800/88/r2/final, 2025.

[8] NVM Express. NVM Express Base Specification: Sanitize Command. https://nvmexpress.org/, 2024.

[9] NVM Express. NVM Express Flexible Data Placement. https://nvmexpress.org/, 2024.

[10] NVM Express. NVM Express Zoned Namespace Command Set Specification. https://nvmexpress.org/, 2024.

[11] S. Oh, J. Kim, S. Han, J. Kim, S. Lee, and S. H. Noh. MIDAS: Minimizing write amplification in log-structured systems through adaptive group number and size configuration. In *Proceedings of the USENIX Conference on File and Storage Technologies*, pages 259–275, 2024.

[12] Open Quantum Safe Project. liboqs: C library for quantum-safe cryptographic algorithms. https://openquantumsafe.org/, 2024.

[13] Open Quantum Safe Project. OpenSSL oqs-provider. https://github.com/open-quantum-safe/oqs-provider, 2024.

[14] RocksDB. ZenFS: Rocksdb file system for zoned block devices. https://github.com/facebook/rocksdb/tree/main/plugin/zenfs, 2024.

[15] Q. Wang, J. Li, P. P. C. Lee, T. Ouyang, C. Shi, and L. Huang. Separating data via block invalidation time inference for write amplification reduction in log-structured storage. In *Proceedings of the USENIX Conference on File and Storage Technologies*, pages 429–444, 2022.

[16] xNVMe Project. xNVMe: Cross-platform libraries and tools for nvme devices. https://xnvme.io/, 2024.